



**Bharati Vidyapeeth's College of Engineering
New Delhi**

LAB MANUAL

ETCS-352

OPERATING SYSTEMS

B.Tech- 6th SEMESTER

INSTITUTE VISION

To be an institute of excellence that provides quality technical education and research to create competent graduates for serving industry and society.

INSTITUTE MISSION

M1: To impart quality technical education through dynamic teaching-learning environment

M2: To promote research and innovations activities which gives opportunities for life-long learning in context of academic and industry.

M3: To build up links with industry-institute through partnerships and collaborative developmental works.

M4: To inculcate work ethics and commitment in graduates for their future endeavors to serve the society.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

VISION OF DEPARTMENT

To develop as a centre of excellence in computer science and engineering education and research so as to produce globally competent professionals with a sense of social responsibility.

MISSION OF DEPARTMENT

Mission: The mission of the CSE Department is to:

M1: Impart technical knowledge in Computer Science and Engineering with the state-of-art infrastructure.

M2: Provide a conducive environment for the holistic development of graduates.

M3: Inculcate leadership qualities, teamwork and strong ethical values among the graduates.

M4: Promote a research culture and industry-academia collaboration to strengthen innovation.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

Following are the three CSE Department Program Educational Objectives (PEOs):

PEO1: To produce graduates with in-depth knowledge of Computer Science and Engineering to contribute towards innovation, research and excellence in higher studies.

PEO2: To inculcate life-long learning skills in graduates enabling them to adapt to changing technologies, modern tools and work in teams.

PEO3: To produce ethically responsible graduates who are involved in transforming the society by providing suitable engineering solutions.

The undergraduate program of CSE is having the following Program Outcomes and Program Specific Outcomes (POs).

PROGRAM OUTCOMES (PO)

- 1. Engineering knowledge:** Apply the knowledge acquired in mathematics, science, engineering for the solution of complex engineering problems.
- 2. Problem analysis:** Identify research gaps, formulate and analyze complex engineering problems drawing substantiated conclusions using basic knowledge of mathematics, natural sciences and engineering sciences.
- 3. Design/development of solutions:** Design solutions for the identified complex engineering problems as well as develop solutions that meet the specified needs for the public health and safety, and the cultural, societal and environmental considerations.
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.
- 5. Modern tool usage:** Work on the latest technologies, resources and software tools including prediction and modelling to complex engineering activities with an understanding of their limitations.

6. **The engineer and society:** Apply the basic acquired knowledge to measure societal, health, safety, legal and cultural issues and identifying the consequential responsibilities relevant to the professional engineering practice.
7. **Environment and Sustainability:** Comprehend the impact of the professional engineering solutions in context of society and environment and demonstrate the need and knowledge for sustainable development
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning.

PROGRAM SPECIFIC OUTCOMES (PSO)

PSO1: Ability to apply fundamentals of computational mathematics and algorithmic formulations to solve the real-time challenges of computer engineering encountered in research and industry.

PSO2: Capability to design and develop software and hardware applications using logical, analytical, and programming skills learnt while also following professional and social ethics.

TABLE OF CONTENTS

S.No.		Page no.
1.	Course details 1.1 Course objective 1.2. Course Out comes 1.3CO-PO/PSO mapping 1.4Evaluation Scheme 1.5Guidelines/Rubrics for continuous assessment 1.6 Lab safety instruction 1.7Instructions for students while writing Experiment in Lab file.	
2.	List of Experiments& CBS	
3.	Experimental Setup details for the course.	
4.	Experiment details	
5.	Course Exit Survey	

1. COURSE DETAILS

1.1 COURSE OBJECTIVE

- *To learn OS commands in C++ programming.
- To implement various CPU Scheduling Algorithms.
- To implement Process Creation and Inter Process Communication.
- To implement Deadlock Avoidance and Deadlock Detection Algorithms.
- To implement Page Replacement Algorithms, File Organization and File Allocation Strategies.

Recommending system/ Software Requirements:

- Turbo C++ compiler in Window operating systems

Subject: OSLab

Subject Code: ETCS 352

1.2 COURSE OUTCOMES:

At the end of the course student will be able to:		PO/ PSO	Bloom Level
ETCS352.1	Study and learn various commands of LINUX operating system	PO1,PO4,PO6, PO7, PO10, PO12, PSO1,PSO2	L1-Remembering L2-Understanding
ETCS352.2	Define and demonstrate various CPU scheduling algorithm	PO1,PO2,PO3, PO4, PO10, PO12, PSO1, PSO	L2-Understanding
ETCS352.3	Implement various pagereplacement algorithm and evaluate their performance	PO1, PO2, PO3, PO4,PO5 PO12, PSO1, PSO2	L5- Evaluating
ETCS352.4	Implement and compare the performanceofvarious memorymanagement algorithms	PO1, PO2, PO3, PO4, PO12, PSO1, PSO2	L4- Analyze
ETCS352.5	Make use of semaphore and build the solution of Reader's Writer's Problem	PO1, PO2, PO3, PO4, PO12, PSO1, PSO2	L3- Applying
ETCS352.6	Constructthe solutionfor Banker'salgorithmfordeadlock avoidance	PO1, PO2, PO3, PO4, PO12, PSO1, PSO2	L6- Creating

	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2
CO1	3			3		2	3			2			3	1
CO2	3	3	3	3						2		2	3	2
CO3	3	3	3	3	2							3	3	2
CO4	3	3	3	3								3	3	2
CO5	3	3	3	3								3	3	2
CO6	3	3	3	3								3	3	2

1=Slightly, 2=moderately, 3=substantially

1.4 EVALUATION SCHEME

	Laboratory	
Components	Internal	External
Marks	40	60
Total Marks	100	

1.5 GUIDELINES FOR CONTINUOUS ASSESSMENT FOR EACH EXPERIMENT

- **Attendance and performance in minimum eight experiments – 30 marks for all semesters**
 - Each Experiment will carry a weight of 15 marks
 - Experiment performance [5 Marks]
 - File [5 Marks]
 - Viva – Voce [5 Marks]
- **2 innovative experiments (Content Beyond syllabus) 10 marks for 1st & 2nd Semester**
- **2 innovative experiments (Content Beyond syllabus) 5 marks for 3rd, 4th, 5th, 6th, 7th, 8th Semester**
- **Viva 5 marks for 3rd, 4th, 5th, 6th, 7th and 8th Semester**

The Rubrics for Experiment execution and Lab file+ viva voce is given below:

Experiment Marks details:

Status	<i>Completed and Executed perfectly</i>	<i>Completed but partially Executing</i>	<i>Logically Incorrect Program or errors</i>	<i>Unacceptable efforts/Absent</i>
Marks	4-5	2-3	1	0

File Marks Details:

Status	<i>File Contents & Checked Timely</i>	<i>File Contents & Checked not Timely (after one week)</i>	<i>File Contents & Checked After two weeks</i>
Marks	4-5	2-3	0-1

Viva-Voce Marks details:

Status	<i>Viva (Good)</i>	<i>Viva (Average)</i>	<i>Viva (Unsatisfactory)</i>
Marks	4-5	1-3	0

Note: Viva Voce Questions for each experiment should be related to Course Outcomes.

1.4 Safety Guidelines/Rules for laboratory (AS PER SUBJECT/ LABORATORY)

DO'S AND DONT'S

1. Maintain discipline inside LAB and in the corridors surrounding it
2. No talking is allowed within the lab.
3. No Noise to be made inside lab and in corridor, in any way.
4. Use of Cell-Phone is not allowed inside the labs and in corridors
5. No Floppy is to be used in the labs
6. All assets inside the lab to be handle with care and are to be protected collectively. Any damage or loss of college property inside lab would be borne by all students equally.
7. No students will be allowed to enter in the lab after 10 minutes of the scheduled time.

1.6Format for students while writing Experiment in Lab file.

Experiment No:

Aim:

Course Outcome:

Software used:

Theory:

Flowchart/Algorithm/Code:

Results:

Expected Outcome attained: YES/NO

2. LIST OF EXPERIMENTS AS PER GGSIPU

Sr. No.	Title of Lab Experiments	CO
1. 1	Write a program to implement CPU scheduling for first come first serve	(CO1)
2. 2	Write a program to implement CPU scheduling for shortest job first.	(CO2)
3. 3	Write a program to perform priority scheduling.	(CO3)
4. 4	Write a program to implement CPU scheduling for Round Robin.	(CO3)
5. 5	Write a program for page replacement policy using a)LRU b)FIFO c)Optima	(CO3)
6. 6	Write a program to implement first fit, best fit and worst fit algorithm form memory management.	(CO4)
7. 7	Write a program to implement reader/writer problem using semaphore.	(CO5)
8. 8	Write a program to implement reader/writer problem using semaphore.	(CO5)

CONTENT BEYOND SYLLABUS

Sr.No	Name of Experiment
1.	Write a program to implement Producer-Consumer problem using semaphores.
2	Write a program to implement Dining-Philosopher problem using semaphores.

3. EXPERIMENTAL SETUP DETAILS FOR THE COURSE

Software Requirements:

C++ , Windows

Minimum Hardware Requirements

Dual Core based PC with 1 GB RAM

4. EXPERIMENTAL DETAILS

PRACTICAL 1

Aim: Write a program to implement CPU scheduling for first come first serve.

Theory: First come, first served (FCFS) is an operating system process scheduling algorithm and a network routing management mechanism that automatically executes queued requests and processes by the order of their arrival.

It uses non-preemptive scheduling in which a process is automatically queued and processing occurs according to an incoming request or process order.

Algorithm:

- 1- Input the processes along with their burst time (bt).
- 2- Find waiting time (wt) for all processes.
- 3- As first process that comes need not to wait so waiting time for process 1 will be 0 i.e. $wt[0]=0$.
- 4- Find waiting time for all other processes i.e. for process $i \rightarrow wt[i]=bt[i-1]+wt[i-1]$.
- 5- Find turn around time = waiting_time + burst_time for all processes.
- 6- Find average waiting time = $\frac{\text{total_waiting_time}}{\text{no_of_processes}}$.
- 7- Similarly, find average turn around time = $\frac{\text{total_turn_around_time}}{\text{no_of_processes}}$.

SampleInputsandOutputs:

```
Enter The Number of Processes To Execute:      3

Enter The Burst Time of Processes:

Process [1]:10
Process [2]:20
Process [3]:30

Process          Burst Time      Waiting Time      Turnaround Time

Process [1]          10.00          0.00          10.00
Process [2]          20.00          10.00         30.00
Process [3]          30.00          30.00         60.00

Average Waiting Time = 13.333333
Average Turnaround Time = 33.333332
```

Viva-Questions:

1. What is CPU utilization?
2. What are the different job scheduling in operating systems?
3. Which scheduling algorithm allocates the CPU first to the process that requests the CPU first?
4. Which of the following statements are true?
 - i) Shortest remaining time first scheduling may cause starvation
 - ii) Preemptive scheduling may cause starvation
 - iii) Round robin is better than FCFS in terms of response time

PRACTICAL -2

Aim: Write a program to implement CPU scheduling for shortest job first.

Theory: Shortest job first (SJF) or shortest job next, is a scheduling policy that selects the waiting process with the smallest execution time to execute next. SJN is a non-preemptive algorithm.

Algorithm:

1- Sort all the processes in increasing order according to burst time. 2- Then simply, apply FCFS.

Sample Inputs and Outputs:

```
Enter the Total Number of Processes:    3
Enter Details of 3 Processes
Enter Arrival Time:      0
Enter Burst Time:       20
Enter Arrival Time:      3
Enter Burst Time:       10
Enter Arrival Time:      7
Enter Burst Time:       30
Average Waiting Time:    11.000000
Average Turnaround Time: 31.000000
```

Viva-Questions:

1. What is real difficulty with SJF in short term scheduling?
2. Preemptive Shortest Job First scheduling is sometimes called _____
3. Which scheduling algorithm gives minimum average waiting time?
4. State true or false: Scheduling is allowing a job to use the processor?
5. What do you mean by Average turnaround time and average waiting time.

PRACTICAL -3

Aim: Write a program to perform priority scheduling.

Theory: Priority scheduling involves priority assignment to every process, and processes with higher priorities are carried out first, whereas tasks with equal priorities are carried out on a first-come-first-served(FCFS)orroundrobinbasis

Algorithm:

- 1- Firstinput theprocesseswiththeir bursttimeandpriority.
- 2- Sort the processes, burst time and priority according to thepriority.
- 3- NowsimplyapplyFCFSalgorithm.

Sample code for priority scheduling:

```
Void priority Scheduling(Processproc[],int n)
{
    // Sort processes by priority
    sort(proc,proc+n, comparison);
    cout<< "Order in which processes gets executed
    \n";for(inti=0;i<n;i++)
        cout<<proc[i].pid<<" ";f
    indavgTime(proc,n);
}
```

SampleInputsandOutputs:

```
Enter Total Number of Processes:      3
Enter Burst Time and Priority For 3 Processes

Process[1]
Process Burst Time:      15
Process Priority:        3

Process[2]
Process Burst Time:      10
Process Priority:        2

Process[3]
Process Burst Time:      90
Process Priority:        1

Process ID      Burst Time      Waiting Time      Turnaround Time
Process[3]      90              0                90
Process[2]      10              90               100
Process[1]      15              100              115

Average Waiting Time:  63.000000
Average Turnaround Time:  101.000000
```

Viva-Questions:

1. In priority scheduling algorithm, when a process arrives at there a dyqueue, its priority is compared with the priority of which process.
2. What is starvation?
3. What is aging?
4. What are the factors for assigning priority to the process?
5. If two processes come on same time, on what criteria will process be chosen for processing?

PRACTICAL -4

Aim: Write a program to implement CPU scheduling for Round Robin.

Theory: Round-robin (RR) is one of the algorithms employed by process and network schedulers in computing. As the term is generally used, time slices (also known as time quanta) are assigned to each process in equal portions and in circular order, handling all processes without priority.

Algorithm:

1. Create an array `rem_bt[]` to keep track of remaining burst time of processes. This array is initialized as a copy of `bt[]` (burst time array).
2. Create another array `wt[]` to store waiting times of processes. Initialize this array as 0.
3. Initialize time: `t=0`
4. Keep traversing all processes while all processes are not done. Do the following for each process `i` if it is not done yet.

```
If rem_bt[i] > quantum
    t = t + quantum
    rem_bt[i] -= quantum;
Else // Last cycle
    for this process t = t + rem_bt[i]
    wt[i];
    wt[i] = t - bt[i]
    rem_bt[i] = 0; // This process is over
```

Sample Inputs and Outputs:

```
Arrival Time: 0
Burst Time: 5

Enter Details of Process[2]:
Arrival Time: 2
Burst Time: 3

Enter Details of Process[3]:
Arrival Time: 3
Burst Time: 2

Enter Details of Process[4]:
Arrival Time: 5
Burst Time: 7

Enter Time Quantum: 2
```

Process ID	Burst Time	Turnaround Time	Waiting Time
Process[3]	2	3	1
Process[2]	3	9	6

Viva-Questions:

1. What do you mean by Time Slice?
2. Time quantum is defined in which scheduling algorithm?
3. What is the limitation of Round Robin Scheduling Algorithm?
4. What are the factors that affect Round Robin Scheduling Algorithm?
5. Round robin scheduling falls under the category of which scheduling? Non-preemptive scheduling or Preemptive scheduling.

PRACTICAL -5

Aim: Write a program for page replacement policy using a)LRU b)FIFO c)Optimal.

Theory: In an operating system that uses paging for memory management, page replacement algorithms are needed to decide which page needed to be replaced when new page comes in. Whenever a new page is referred and not present in memory, page fault occurs and Operating System replaces one of the existing pages with newly needed page. Different page replacement algorithms suggest different ways to decide which page to replace. The target for all algorithms is to reduce number of page faults.

Page Fault – A page fault is a type of interrupt, raised by the hardware when a running program accesses a memory page that is mapped into the virtual address space, but not loaded in physical memory.

Least Recently Used (LRU)–

In this algorithm, page will

be replaced which is least recently used. **First In First Out (FIFO)–**

This is the simplest page replacement algorithm. In this algorithm, operating system keeps track of all pages in the memory in a queue, oldest page is in the front of the queue. When a page needs to be replaced, page in the front of the queue is selected for removal.

Optimal Page replacement–

In this algorithm, pages are replaced which are not used for the longest duration of time in the future.

Algorithm (LRU):

Let capacity be the number of pages that memory can hold. Let set be the current set of pages in memory.

1- Start traversing the pages.

If set holds less pages than capacity.

Insert page into the set one by one until the size of set reaches capacity or all page requests are processed.

Simultaneously maintain the recent occurred index of each page in a map called indexes.

Increment page fault

Else

If current page is present in set, do nothing. Else

Find the page in the set that was least recently used. We find it using index array.

We basically need to replace the page with minimum index.

Replace the found page with current page.

Increment page faults.

Update index of current page.

2. Return page faults.

Algorithm(FIFO):

1- Start traversing the pages.

If set holds less pages than capacity.

Insert page into the set one by one until the size of set reaches capacity or all page requests are processed.

Simultaneously maintain the pages in the queue to perform FIFO.

Increment page fault

Else

If current page is present in set, do nothing. Else

Remove the first page from the queue as it was the first to be entered in the memory

Replace the first page in the queue with the current page in the string.

Store current page in the queue.

Increment page faults.

2. Return page faults.

Pseudocode (Optimal Page replacement):

```
void optimalPage(int pg[], int pn, int fn)
{
    // Create an array for given number of
    // frames and initialize it

    a.empty().vector<int> fr;

    // Traverse through page reference array
    // and check for miss and hit. int

    hit=0;

    for(int i=0; i<pn; i++){
        // Page found in a frame :

        HIT if (search(pg[i], fr)){

            hit++; continue;

        }
        // Page not found in a frame: MISS
        // If there is space available in frames. if (fr

        .size()<fn)

            fr.push_back(pg[i]);

        // Find the page to be replaced. else {
```

```

        int j = predict(pg, fr, pn,
        i+1); fr[j] = pg[i];
    }
}
cout << "No. of hits = " << hit << endl;
cout << "No. of misses = " << pn-hit << endl;
}

```

Optimal Page replacement example

In this algorithm, pages are replaced which are not used for the longest duration of time in the future.

Let us consider page reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 and 4 page slots.

Initially all slots are empty, so when 7 0 1 2 are allocated to the empty slots → 4 Page faults

0 is already there so → 0 Page fault.

when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future. → 1 Page fault.

0 is already there so → 0 Page fault..

4 will take place of 1 → 1 Page Fault.

Now for the further page reference string —
→ 0 Page fault because they are already available in the memory.

Example-2, Let's have a reference string: a, b, c, d, c, a, d, b, e, b, a, b, c, d and the size of the frame be 4.

Time Req		1	2	3	4	5	6	7	8	9	10	11	12	13	14
Page frames		a	b	c	d	c	a	d	b	e	b	a	b	c	d
0	a	a	a	a	a	a	a	a	a	a	e	e	e	e	d
1	b		b	b	b	b	b	b	b	b	b	a	a	a	a
2	c			c	c	c	c	c	c	c	c	c	b	b	b
3	d				d	d	d	d	d	e	d	d	d	c	c
FAULTS		x	x	x	x					x					x

There are 6 page faults using optimal algorithm.

Optimal page replacement is perfect, but not possible in practice as operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analyzed against it.

Sample Inputs and

Outputs: Example 1 (LRU):

Enter number of pages: 10

Enter the reference string: 7

5 9 4 3 7 9 6 2 1 Enter number of frames: 3

7

7	5	
7	5	9
4	5	9
4	3	9
4	3	7
9	3	7
9	6	7
9	6	2
1	6	2

The number of page faults is 10

Example 2 (LRU):

Enter number of frames: 3

Enter number of pages: 6

Enter reference string: 5 7 5 6 7 3

5-1-1

5-7-1

5-7-1

5-7-6

5-7-6

3-7-6

Total Page Faults = 4

Example(FIFO)

ENTER THE NUMBER OF PAGES: 20

ENTER THE PAGE NUMBER: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 ENTER THE

NUMBER OF FRAMES: 3

<u>refstring</u>	<u>page frames</u>
------------------	--------------------

7	7	-1	-1
0	7	0	-1
1	7	0	1
2	2	0	1
0			
3	2	3	1
0	2	3	0
4	4	3	0
2	4	2	0
3	4	2	3
0	0	2	3
3			
2			
1	0	1	3
2	0	1	2
0			
1			
7	7	1	2
0	7	0	2
1	7	0	1

PageFaults15

Example1(Optimalpagereplacement output)

Enter number of frames:

3Enter number of pages:10

Enter page reference string:2342137543

2-1-1

23-1

234

234

134

134

734

534

534

534

Example2 (Optimalpagereplacement output)

Sample Input: Number of frames, fn=3

ReferenceString, pg[]={7,0, 1, 2,0, 3, 0,4, 2, 3,0,3, 2, 1,2, 0, 1,7, 0, 1};

Sample Output:

No.of hits=11 No.of misses

=9

Sample Input: Number of frames, fn=4

ReferenceString, pg[]={7,0, 1, 2, 0,3, 0, 4,2, 3,0, 3,

2}; Sample Output: No.of hits =7

No.of misses =6

Viva-Questions:

1. Define Demand Paging, Page fault interrupt, and Thrashing?
2. Under what circumstances do page faults occur? Describe the action taken by the operating system when a page fault occurs?
3. What is the cause of thrashing? How does the system detect thrashing? Once it detects thrashing, what can the system do to eliminate this problem?
4. What is a page and what is a frame. How are they related?
5. What do you mean by Belady's anomaly?
6. Compare LRU, FIFO and Optimal page replacement algorithm.

PRACTICAL-6

Aim: Write a program to implement first fit, best fit and worst fit algorithm for memory management.

Theory: First fit: In the first fit approach is to allocate the first free partition or hole large enough which can accommodate the process. It finishes after finding the first suitable free partition.

Best Fit: The best fit deals with allocating the smallest free partition which meets the requirement of the requesting process. This algorithm first searches the entire list of free partitions and considers the smallest hole that is adequate. It then tries to find a hole which is close to actual process size needed.

Worst fit: In worst fit approach is to locate largest available free portion so that the portion left will be big enough to be useful. It is the reverse of best fit.

Algorithm(Firstfit):

- 1- Input memory blocks with size and processes with size.
- 2- Initialize all memory blocks as free.
- 3- Start by picking each process and check if it can be assigned to current block.
- 4- If size of process $\leq$ size of block, if yes then assign and check for next process.
- 5- If not then keep checking the further blocks.

Algorithm(Best fit):

- 1- Input memory blocks and processes with sizes.
- 2- Initialize all memory blocks as free.
- 3- Start by picking each process and find the minimum block size that can be assigned to current process i.e., find $\min(\text{blockSize}[1], \text{blockSize}[2], \dots, \text{blockSize}[n]) > \text{processSize}[\text{current}]$, if found then assign it to the current process.
- 5- If not then leave that process and keep checking the further processes.

Algorithm(Worst fit):

- 1- Input memory blocks and processes with sizes.
- 2- Initialize all memory blocks as free.
- 3- Start by picking each process and find the minimum block size that can be assigned to current process
i.e., $\text{findmin}(\text{blockSize}[1], \text{blockSize}[2], \dots, \text{blockSize}[n]) >$
- 4- $\text{processSize}[\text{current}]$, if found then assign it to the current process.
- 5- If not then leave that process and keep checking the further processes.

Sample Inputs and Outputs:

Example(First fit):

Sample Input: $\text{blockSize}[] = \{100, 500, 200, 300, 600\};$

$\text{processSize}[] = \{212, 417, 112,$

$426\};$ Sample Output:

ProcessNo.	ProcessSize	Blockno.
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

Example(Best fit):

Sample Input: $\text{blockSize}[] = \{100, 500, 200, 300, 600\};$

$\text{processSize}[] = \{212, 417, 112,$

$426\};$ Sample Output:

ProcessNo.	ProcessSize	Blockno.
1	212	4
2	417	2
3	112	3

4 426 5

Example(Worstfit):

SampleInput: blockSize[] = {100,500,200,300,600};

processSize[] = {212, 417, 112,

426};Sample Output:

ProcessNo.	ProcessSize	Blockno.
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

Viva-Questions:

1. Difference between Logical and Physical Address Space?
2. Describe first-fit, best-fit strategies and worst-fit strategies for disk space allocation, with their merits and demerits.
3. What is the impact of fixed partitioning on fragmentation?
4. In fixed-sized partition, the degree of multiprogramming is bounded by _____
5. The first-fit, best-fit and worst-fit are strategies to select a

PRACTICAL -7

Aim: Write a program to implement reader/writer problem using semaphore.

Theory: Readers-Writer sproblem

Problem parameters:

- One set of data is shared among a number of processes
- Once a writer is ready, it performs its write. Only one writer may write at a time
- If a process is writing, no other process can read it
- If at least one reader is reading, no other process can write
- Readers may not write and only read

Solution when Reader has the Priority over Writer

Here priority means, no readers should wait if the share is currently opened for reading.

Three variables are used: mutex, wrt, readcnt to implement solution

1. semaphore mutex, wrt; // semaphore mutex is used to ensure mutual exclusion when readcnt is updated i.e. when any reader enters or exits from the critical section and semaphore wrt is used by both readers and writers
2. int readcnt; // readcnt tells the number of processes performing read in the critical section, initially 0

Writer process:

1. Writer requests the entry to critical section.
2. If allowed i.e. wait() gives a true value, it enters and performs the write. If not allowed, it keeps on waiting.
3. It exits the critical section.

Reader process:

1. Reader requests the entry to critical section.
2. If allowed:
 - it increments the count of number of readers inside the critical section. If this reader is the first reader entering, it locks the **wrt** semaphore to restrict the entry of writers if any reader is inside.
 - It then, signals mutex as any other reader is allowed to enter while others are already reading.
 - After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore "wrt" as now, writer can enter the critical section.
3. If not allowed, it keeps on waiting.

Pseudo code:

Writer process:

```
do{  
    //writerrequestsforcriticalsectionwa  
    it(wrt);  
  
    //performsthewrite  
  
    //leavesthecriticalsections  
    gnal(wrt);  
} while(true);
```

Readerprocess:

```
do{  
    //  
    Readerwantstoenterthecriticalsectionwait(m  
   utex);  
    //Thenumberofreadershasnowincreasedby1readc  
    nt++;  
    //thereisatleastonereaderinthecriticalsection  
    //thisensurenowritercanenterifthereisevenonereader  
  
    // thus we give preference to readers  
    hereif(readcnt==1)  
        wait(wrt);  
  
    //other readerscanenterwhilethiscurrentreaderisinside  
  
    //thecriticalsections  
    gnal(mutex);  
    //currentreaderperformsreadingherewa  
    it(mutex);//areaderwantstoleave  
    readcnt--;  
  
    //thatis,noreaderisleftinthecriticalsection,if(re  
    dcnt==0)  
        signal(wrt);    //writerscanenter  
  
    signal(mutex);//readerleaves  
} while(true);
```

Sample Inputs and Outputs:

```
File Edit View Terminal Help
student@cc-02-desktop:~$ cd 004
student@cc-02-desktop:~/004$ cc readerw.c -lpthread -o readerw
student@cc-02-desktop:~/004$ ./readerw

value of shared variable is 0
W :written =0
W going to sleep..
R :read=0 Reader no.=1
Reader1:going to sleep...
R :read=0 Reader no.=2
Reader2:going to sleep...
R :read=0 Reader no.=1
Reader1:going to sleep...
R :read=0 Reader no.=2
Reader2:going to sleep...
R :read=0 Reader no.=1
Reader1:going to sleep...
R :read=0 Reader no.=2
Reader2:going to sleep...
R :read=0 Reader no.=1
Reader1:going to sleep...
R :read=0 Reader no.=2
Reader2:going to sleep...
R :read=0 Reader no.=1
Reader1:going to sleep...
R :read=0 Reader no.=2
Reader2:going to sleep...Reader no.=1 livingReader no.=2 living
W :written =2
W going to sleep..
W :written =4
W going to sleep..Writer is livingstudent@cc-02-desktop:~/004$
```

Viva-Questions:

1. Define the critical section problem and explain the necessary characteristics of a correct solution.
2. What do understand by Race Condition?
3. Define semaphore and its limitations.
4. Define monitor.
5. What are classical problems of process synchronization?

PRACTICAL-8

Aim: Write a program to implement Banker's algorithm for dead lock avoidance.

Theory: Banker's algorithm is a deadlock avoidance algorithm. It is named so because this algorithm is used in banking systems to determine whether a loan can be granted or not.

Consider there are n account holders in a bank and the sum of the money in all of their accounts is S . Every time a loan has to be granted by the bank, it subtracts the loan amount from the total money the bank has. Then it checks if that difference is greater than S . It is done because, only then, the bank would have enough money even if all the n account holders draw all their money at once.

Banker's algorithm works in a similar way in computers. Whenever a new process is created, it must exactly specify the maximum instances of each resource type that it needs.

Let us assume that there are n processes and m resource types. Some data structures are used to implement the banker's algorithm. They are:

1. **Available:** It is an array of length m . It represents the number of available resources of each type. If $Available[j] = k$, then there are k instances available of resource type R_j .
2. **Max:** It is an $n \times m$ matrix which represents the maximum number of instances of each resource that a process can request. If $Max[i][j] = k$, then the process P_i can request at most k instances of resource type R_j .
3. **Allocation:** It is an $n \times m$ matrix which represents the number of resources of each type currently allocated to each process. If $Allocation[i][j] = k$, then process P_i is currently allocated k instances of resource type R_j .
4. **Need:** It is an $n \times m$ matrix which indicates the remaining resource needs of each process. If $Need[i][j] = k$, then process P_i may need k more instances of resource type R_j to complete its task.

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

ResourceRequestAlgorithm:

This describes the behaviour of the system when a process makes a resource request in the form of a request matrix. The steps are:

1. If number of requested instances of each resource is less than the need (which was declared previously by the process), go to step 2.
2. If number of requested instances of each resource type is less than the available resources of each type, go to step 3. If not, the process has to wait because sufficient resources are not available yet.
3. Now, assume that the resources have been allocated. Accordingly do,

$Available = Available -$

$Request_i Allocation_i = Allocation_i +$

$Request_i Need_i = Need_i - Request_i$

This step is done because the system needs to assume that resources have been allocated. So there will be less resources available after allocation. The number of allocated instances will increase. The need of the resources by the process will reduce. That's what is represented by the above three operations.

After completing the above three steps, check if the system is in safe state by applying the safety algorithm. If it is in safe state, proceed to allocate the requested resources. Else, the process has to wait longer.

SafetyAlgorithm:

1. Let $Work$ and $Finish$

be vectors of length m and n , respectively. Initially, $Work = Available$

$Finish[i] = false$ for $i = 0, 1, \dots, n-1$.

This means, initially, no process has finished and the number of available resources is represented by the $Available$ array.

2. Find an index i such that
both $Finish[i] == false$

$Need_i \leq Work$

If there is no such process present, then proceed to step 4.

It means, we need to find an unfinished process whose need can be satisfied by the available resources. If no such process exists, just go to step 4.

3. Perform the following:

Work = Work + Allocation; F_i

finish[i] = true;

Go to step 2.

When an unfinished process is found, then the resources are allocated and the process is marked finished. And then, the loop is repeated to check the same for all other processes.

4. If $Finish[i] == \text{true}$ for all i , then the system is in a safe state.

That means if all processes are finished, then the system is in a safe state.

Sample Inputs and Outputs: E

Example 1 for safe

sequence: Input:

Total Resources	R1	R2	R3
	10	5	7

Process	Allocation			Max		
	R1	R2	R3	R1	R2	R3
P1	0	1	0	7	5	3
P2	2	0	0	3	2	2
P3	3	0	2	9	0	2
P4	2	1	1	2	2	2

Output: Safe sequences are: P

2 → P4 → P1 → P3

P2 → P4 → P3 → P1

P4 → P2 → P1 → P3

P4 → P2 → P3 → P1

There are total 4 safe-sequences

Example2

For non-safe state

Enter the number of process:3

Enter the 0th process allocated resources:1 2

3Enter the 0th process maximum resources:4 5

6Enter the 1th process allocated resources:3 4

5Enter the 1th process maximum resources:6 7

8Enter the 2th process allocated resources:1 2

3Enter the 2th process maximum resources:3 4

5Enter the available vector:101211

The state sequence:SYSTEM IS NOT IN A SAFE STATE

Viva-Questions:

1. What is resource allocation graph?
2. What are the deadlock avoidance algorithms?
3. Recovery from Deadlock?
4. When does deadlock happen? How does Banker's algorithm avoid the deadlock condition
5. What necessary condition can lead to a deadlock situation in a system?
6. What factors determine whether a detection-algorithm must be utilized in a deadlock avoidance system?
7. What is a Safe State and its use in deadlock avoidance?
8. What are the Methods for Handling Deadlocks?

CONTENT BEYOND SYLLABUS –I

AIM: To write a C program to implement the Producer & consumer Problem (Semaphore)

DESCRIPTION: Producer-consumer problem, is a common paradigm for cooperating processes. A producer process produces information that is consumed by a consumer process. One solution to the producer-consumer problem uses shared memory. To allow producer and consumer

processes to run concurrently, there must be available a buffer of items that can be filled by the producer and emptied by the consumer. This buffer will reside in a region of memory that is shared by the producer and consumer processes. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

ALGORITHM:

Step 1: The Semaphore mutex, full & empty are initialized.

Step 2: In the case of producer process i) Produce an item in to temporary variable. ii) If there is empty space in the buffer check the mutex value for enter into the critical section. iii) If the mutex value is 0, allow the producer to add value in the temporary variable to the buffer.

Step 3: In the case of consumer process i) It should wait if the buffer is empty ii) If there is any item in the buffer check for mutex value, if the mutex==0, remove item from buffer iii) Signal the mutex value and reduce the empty value by 1. iv) Consume the item.

Step 4: Print the result

PROGRAM :

```
#define BUFFERSIZE 10
int mutex,n,empty,full=0,item,item1;
int buffer[20];
int in=0,out=0,mutex=1;
void wait(int s)
{
while(s<0)
{
printf("\nCannot add an item\n");
exit(0);
}
s--;
}
void signal(int s)
{
s++;
}
void producer()
{
do
{
wait (empty);
wait(mutex);
printf("\nEnter an item:");
scanf("%d",&item);
buffer[in]=item; in=in+1;
```

```

signal(mutex);
signal(full);
}
while(in<n);
}
void consumer()
{
do
{
wait(full);
wait(mutex);
item1=buffer[out];
printf("\nConsumed item =%d",item1); out=out+1;
signal(mutex);
signal(empty);
} while(out<n);
} void main() {
printf("Enter the value of n:"); scanf("%d",&n); empty=n;
while(in<n)
producer();
while(in!=out) consumer();
}

```

OUTPUT: \$ cc prco.c \$ a.out

Enter the value of n :

2 Enter the item:

3 2

Enter the item:5

Enter the item:9

consumed item=2

consumed item=5

consumed item=9

RESULT: Thus the program for solving producer and consumer problem using semaphore was executed successfully.

VIVA QUESTIONS

1. Define Semaphore?
2. What is use of wait and signal functions?
3. What is mutual exclusion?
4. Define producer consumer problem?
5. What is the need for process synchronization?
6. Discuss the consequences of considering bounded and unbounded buffers in producer-consumer problem?
7. Can producer and consumer processes access the shared memory concurrently? If not which technique provides such a benefit?

CONTENT BEYOND SYLLABUS -2

DINING-PHILOSOPHERS PROBLEM.

AIM: Write a C program to simulate the concept of Dining-Philosophers problem.

DESCRIPTION : The dining-philosophers problem is considered a classic synchronization problem because it is an example of a large class of concurrency-control problems. It is a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner. Consider five philosophers who spend their lives thinking and eating. The philosophers share a circular table surrounded by five chairs, each belonging to one philosopher. In the center of the table is a bowl of rice, and the table is laid with five single chopsticks. When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her (the chopsticks that are between her and her left and right neighbors). A philosopher may pick up only one chopstick at a time. Obviously, she cannot pick up a chopstick that is already in the hand of a neighbor. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chopsticks. When she is finished eating, she puts down both of her chopsticks and starts thinking again. The dining-philosophers problem may lead to a deadlock situation and hence some rules have to be framed to avoid the occurrence of deadlock.

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>

sem_t room;
sem_t chopstick[5];

void * philosopher(void *);
void eat(int);
int main()
{
    int i,a[5];
    pthread_t tid[5];

    sem_init(&room,0,4);

    for(i=0;i<5;i++)
        sem_init(&chopstick[i],0,1);

    for(i=0;i<5;i++){
        a[i]=i;
        pthread_create(&tid[i],NULL,philosopher,(void *)&a[i]);
    }
```

```

for(i=0;i<5;i++)
pthread_join(tid[i],NULL);
}

void * philosopher(void * num)
{
int phil=*(int *)num;

sem_wait(&room);
printf("\nPhilosopher %d has entered room",phil);
sem_wait(&chopstick[phil]);
sem_wait(&chopstick[(phil+1)%5]);

eat(phil);
sleep(2);
printf("\nPhilosopher %d has finished eating",phil);

sem_post(&chopstick[(phil+1)%5]);
sem_post(&chopstick[phil]);
sem_post(&room);
}

void eat(int phil)
{
printf("\nPhilosopher %d is eating",phil);
}

```

RESULT: Thus the program to implement the dining Philosopher was executed and verified.

VIVA QUESTIONS: 1. Differentiate between a monitor, semaphore and a binary semaphore? 2. Define clearly the dining-philosophers problem? 3. Identify the scenarios in the dining-philosophers problem that leads to the deadlock situations?

ASSIGNMENT: 1. Write a C program to simulate readers-writers problem using monitors?

Department of Computer Science and Engineering
Course Exit Survey
2020- 2021

Subject Name: operating system lab

Semester: 5th

Subject Code: ETCS 352

Please rate how well you understood the course (Tick the most appropriate option)
(1 – Poor, 2- Good, 3- Excellent)

- ETCS352.1 Learnt various commands of LINUX operating system?
☐ 1. ☐ 2. ☐ 3.
- ETCS352.2 Rate your capability to evaluate CPU efficiency in terms of turnaround time, waiting time and throughput for various CPU scheduling algorithm?
☐ 1. ☐ 2. ☐ 3.
- ETCS352.3 Implemented various page replacement algorithm and evaluated their performance on the basis of number of hits and misses?
☐ 1. ☐ 2. ☐ 3.
- ETCS352.4 Compared the performance of various memory management algorithms and able to evaluate the best memory management algorithm?
☐ 1. ☐ 2. ☐ 3.
- ETCS352.5 Rate your ability to devise the solution of reader's writer's problem using semaphores.
☐ 1. ☐ 2. ☐ 3.
- ETCS352.6 To what extent are you able devise the safe sequence for deadlock avoidance using Banker's algorithm?
☐ 1. ☐ 2. ☐ 3.

Suggestions to improve the teaching methodology:

Overall, how do you rate your understanding of the subject (tick whichever is applicable)
1. Below 50%. 2. 50%-70%. 3. 70%-90% 4. Above 90%

Enrolment number

Name of student

Signature